

Entendendo o JavaScript: Síncrono, Assíncrono e Callbacks

1. JavaScript "single-threaded"

Significa que o motor do JavaScript possui apenas **uma única trilha de execução** (uma única "mão" para trabalhar). Ele processa apenas uma instrução de código por vez na pilha principal, sem executar tarefas em paralelo de forma nativa no mesmo contexto.

2. Ordem de execução

A mensagem "**Teste**" apareceria primeiro. Mesmo com 0ms, o `setTimeout` é enviado para a fila de tarefas assíncronas (Task Queue). O JavaScript só olha para essa fila quando terminar de executar todo o código síncrono atual.

3. Código do `setTimeout`

JavaScript

```
setTimeout(() => {  
  console.log("Assíncrono");  
}, 3000);
```

4. Por que o "Callback Hell" é má prática?

- **Dificuldade de Leitura:** O código cresce horizontalmente (formato de pirâmide), tornando-o confuso.
- **Manutenção Complexa:** Alterar uma lógica no meio da corrente pode quebrar todas as outras.
- **Tratamento de Erros:** É extremamente difícil gerenciar erros e exceções de forma limpa em múltiplos níveis de aninhamento.
- **Modernidade:** Existem alternativas muito mais elegantes e legíveis, como **Promises** e **Async/Await**.

5. O Componente Responsável

O **Event Loop** (Laço de Eventos). Ele atua como um vigia, verificando constantemente se a *Call Stack* (Pilha de Execução) está vazia para então mover as funções que estão aguardando na *Task Queue* para serem executadas.

Modernizando o JavaScript: Entendendo as Promises

1. Os três estados de uma Promise

- **Pending** (Pendente): A operação começou, mas ainda não terminou.
- **Fulfilled** (Resolvida): A operação foi concluída com sucesso.
- **Rejected** (Rejeitada): A operação falhou ou ocorreu um erro.

2. Quando o `.catch()` é invocado?

Ele é invocado exclusivamente quando a Promise assume o estado de **Rejected** (Rejeitada), servindo para capturar e tratar o erro ou a falha da operação assíncrona.

Aqui está a resposta direta para a sua pergunta:

3. Modificando o Código

Para buscar o ID 2, você só precisa alterar o número passado dentro dos parênteses na hora de chamar a função. A parte do consumo da Promise ficaria assim:

JavaScript

```
console.log("Iniciando busca...");
```

```
buscarUsuario(2) // <-- Alteramos o ID de 1 para 2 aqui
  .then(dados => console.log(`Usuário encontrado: ${dados.nome}`))
  .catch(erro => console.error(`Falha: ${erro}`));
```

Comportamento do Console de Erro

Ao rodar esse código modificado, o comportamento será exatamente este:

1. O console imprimirá imediatamente: "Iniciando busca...".
2. O sistema aguardará 1,5 segundos (devido ao `setTimeout` que simula o tempo de resposta do servidor).
3. Dentro da função, a verificação `if(id === 1)` será **falsa** (pois agora o id é 2).
4. O código cairá no bloco `else` e executará a função `reject()`. Isso muda o estado da Promise para **Rejected** (Rejeitada).
5. O método `.then()` será completamente ignorado.
6. O método `.catch()` vai "capturar" a rejeição e o console exibirá, em formato de erro (geralmente em vermelho nos navegadores), a seguinte mensagem:

Falha: Erro 404: Usuário não encontrado no banco de dados.

4. Promise resolvida imediatamente

JavaScript

```
Promise.resolve("Olá, Mundo");
```

5. Falta de internet durante um *fetch*

A Promise assumirá o estado **Rejected** (Rejeitada). O `fetch` rejeita a promessa automaticamente quando ocorre uma falha na rede ou a conexão é interrompida.

A Evolução da Leitura: Async/Await e Try/Catch

1. Onde declarar o `async`?

Você deve colocar a palavra `async` obrigatoriamente **antes da declaração da função** que irá conter o `await`. Pode ser em uma função tradicional (`async function buscar() {...}`) ou em uma *arrow function* (`const buscar = async () => {...}`). (Nota: navegadores modernos permitem "top-level await" em módulos específicos, mas a regra geral para escopos locais exige a função `async`).

2. O que acontece sem o `try/catch` diante de uma falha?

A função `async` vai retornar uma **Promise rejeitada silenciosamente**. Se esse erro não for capturado em nenhum outro lugar do código, ele vai gerar um aviso crítico no console chamado **"Unhandled Promise Rejection"** (Rejeição de Promessa Não Tratada), o que pode quebrar a aplicação ou deixá-la em um estado inconsistente.

3. Removendo o `await`.

Se você remover a palavra `await`, o comportamento do código mudará drasticamente. O valor da variável `usuario` não será mais o objeto com os dados da pessoa, mas sim o próprio objeto da Promise em seu estado inicial (Pending).

O que acontece na prática

Ao remover o `await`, você retira a instrução de "pausa" do Maestro. O interpretador não vai mais esperar a função terminar de buscar os dados no "banco de dados" (o tempo de 1,5 segundos estipulado pelo `setTimeout`). Ele passará reto para a próxima linha de forma síncrona.

O código ficará assim:

JavaScript

```
const usuario = buscarUsuario(id); // Sem o await!
```

O Comportamento do Console

Se você rodar esse código no console passando o ID 1, esta será a sequência exata de eventos:

1. Renderizando esqueleto da tela... (Executa normalmente).
2. A função `buscarUsuario(1)` é acionada e retorna a Promise pendente imediatamente para a variável `usuario`.
3. O código não espera. Ele vai direto para a próxima linha e imprime:
Sucesso: Atualizando DOM com o nome `undefined`
Por que `undefined`? Porque a variável `usuario` agora guarda um objeto do tipo `Promise`, e uma Promise não possui a propriedade `.nome`. Os dados reais ainda não chegaram.
4. O bloco não encontrou nenhum erro imediato, então ignora o `catch`.
5. O bloco `finally` é executado imediatamente:
Ocultando animação global de carregamento...

Resumo da Ópera: Sem o `await`, você tentou montar o hambúrguer antes da carne ficar pronta. O sistema seguiu o fluxo em frações de milissegundos usando a promessa (o `pager`) no lugar da resposta real, resultando em dados indefinidos (`undefined`) na interface.

4. A utilidade do bloco `finally`

O `finally` é a **etapa garantida**. Ele executa o código que está dentro dele independentemente do que aconteceu antes — seja um sucesso absoluto no `try` ou um desastre total no `catch`. É extremamente útil para "limpar a bagunça", como por exemplo, esconder o ícone de "Carregando..." na tela do usuário, pois isso deve acontecer quer a busca tenha dado certo ou não.

5. O caminho da execução com ID 5 (Cenário de Erro)

Se chamarmos a operação com `await buscarUsuario(5)` dentro de um bloco `try/catch`, o mapa mental do caminho é este:

1. O interpretador entra no bloco `try` otimista.
2. Ele executa a função com ID 5 e **pausa** no `await`.
3. A função processa e percebe que o ID não é o esperado (falso para `id === 1`), acionando a rejeição da Promise.
4. O `await` recebe o erro. A execução do bloco `try` é **imediatamente abortada** (qualquer console de sucesso abaixo dele é ignorado).
5. O fluxo pula direto para o bloco `catch`, que "segura" o erro e executa a contingência (exibindo a mensagem de falha no console).